

VeloScope

Rapport d'analyse et de conception

API de collecte, d'historisation et d'analyse de données de vélos en libre-service

Technologies	Python · FastAPI · PostgreSQL · pytest · ruff · Git
Source GBFS	Vélib' Métropole
Périmètre	F1 à F5 · extensions optionnelles en perspectives

Version détaillée — analyse, conception, architecture, données, algorithmes, tests et planification

1. Synthèse du projet

VeloScope est une API destinée à collecter périodiquement les données d'un service de vélos en libre-service, à les historiser dans PostgreSQL puis à fournir des fonctionnalités de consultation, d'analyse de fiabilité et de recommandation. Le problème central consiste à transformer un flux d'état courant en un historique exploitable dans le temps.

L'architecture sépare les responsabilités : les contrôleurs gèrent HTTP et la validation, les services portent la logique métier, les DAO isolent SQL et l'adaptateur GBFS encapsule la dépendance au fournisseur externe. Cette organisation améliore la testabilité et réduit le couplage.

Décision structurante : chaque observation est enregistrée comme un snapshot horodaté. Les anciennes observations ne sont jamais écrasées ; elles constituent la base des statistiques, de la fiabilité et des recommandations.

2. Analyse fonctionnelle

2.1 Acteurs

Utilisateur : inscription, connexion, consultation des stations, statistiques, recommandations et favoris. Administrateur : mêmes fonctions, avec gestion des utilisateurs et supervision des collectes.

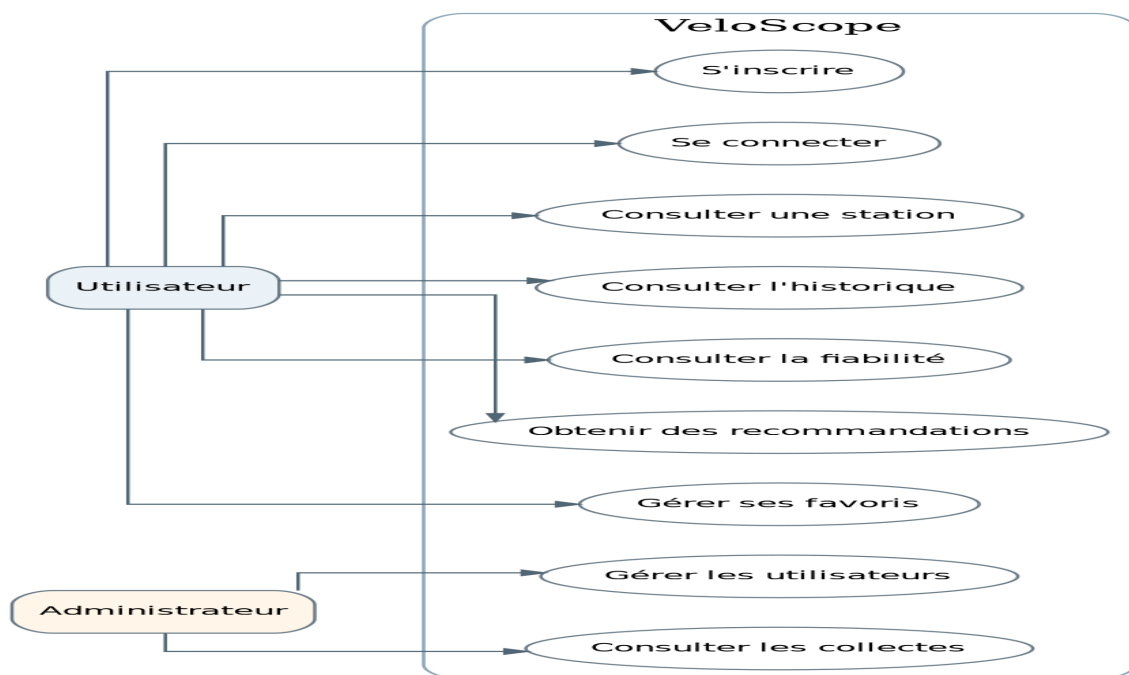


Figure 1 — Cas d'utilisation principaux.

2.2 Règles métier

- EMPTY si bikes = 0 ; FULL si docks = 0 ; FUNCTIONAL sinon.
- Un snapshot correspond à un état observé à un instant donné et ne remplace aucune observation antérieure.
- Une recommandation combine plusieurs critères ; la distance seule ne suffit pas.
- Les données externes sont validées et normalisées avant persistance.

2.3 Scénarios représentatifs

Consultation : Client → Controller → Service → DAO → PostgreSQL, puis retour sous forme JSON.

Collecte : Scheduler → CollectionService → GBFSProvider → API GBFS → normalisation → SnapshotDAO
→ PostgreSQL.

3. Étude de la source GBFS

Flux	Données exploitées	Usage
station_information	identifiant, nom, latitude, longitude, capacité	référentiel
station_status	vélos, bornes, e-bikes, état, date	snapshots et état courant

3.1 Abstraction du fournisseur

GBFSProvider joue le rôle d'adaptateur. Il réalise les appels externes, vérifie les données et les convertit vers un modèle interne stable. Si le fournisseur change un nom de champ ou une structure, la modification reste localisée dans ce composant.

3.2 Gestion des erreurs

- Timeout ou indisponibilité : collecte en échec, historique précédent conservé.
- JSON invalide : payload rejeté.
- Champ obligatoire absent : observation non persistée.
- Valeur incohérente : rejet ou signalement.

4. Architecture logicielle

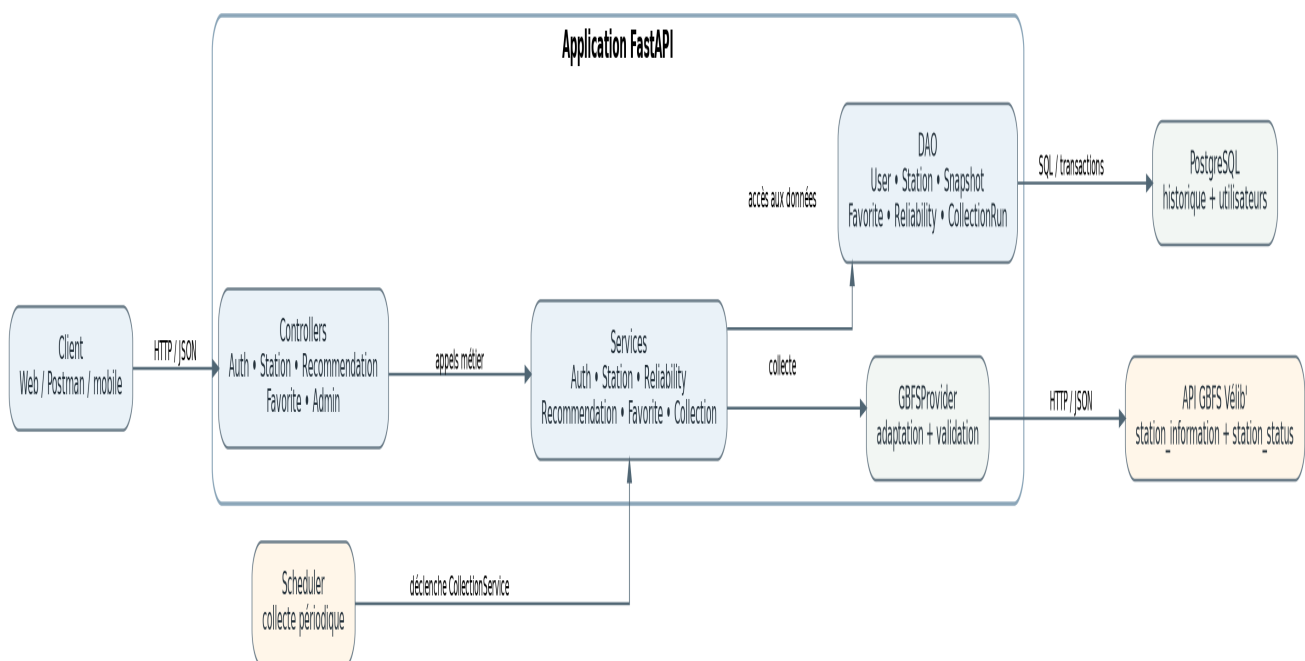


Figure 2 — Architecture globale de VeloScope.

La chaîne principale est Contrôleur → Service → DAO. Le scheduler déclenche la collecte et GBFSProvider isole l'intégration externe. PostgreSQL reste la source de vérité pour les données historisées.

La séparation empêche notamment qu'une route HTTP contienne du SQL ou qu'un service métier connaisse les détails du protocole externe. Elle permet aussi de tester les services avec des mocks.

5. Conception par packages

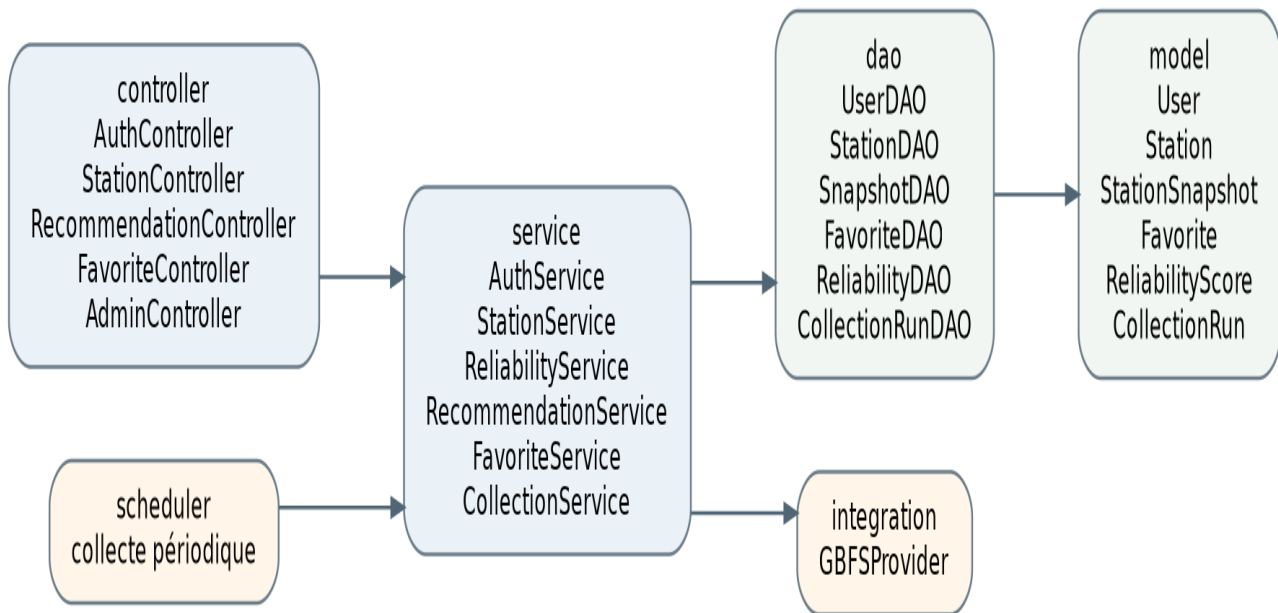


Figure 3 — Organisation des packages.

Controllers : Auth, Station, Recommendation, Favorite, Admin. Services : Auth, Station, Reliability, Recommendation, Favorite, Collection. DAO : User, Station, Snapshot, Favorite, Reliability, CollectionRun. Model : entités métier. Integration : GBFSPProvider. Scheduler : déclenchement périodique.

6. Modèle objet

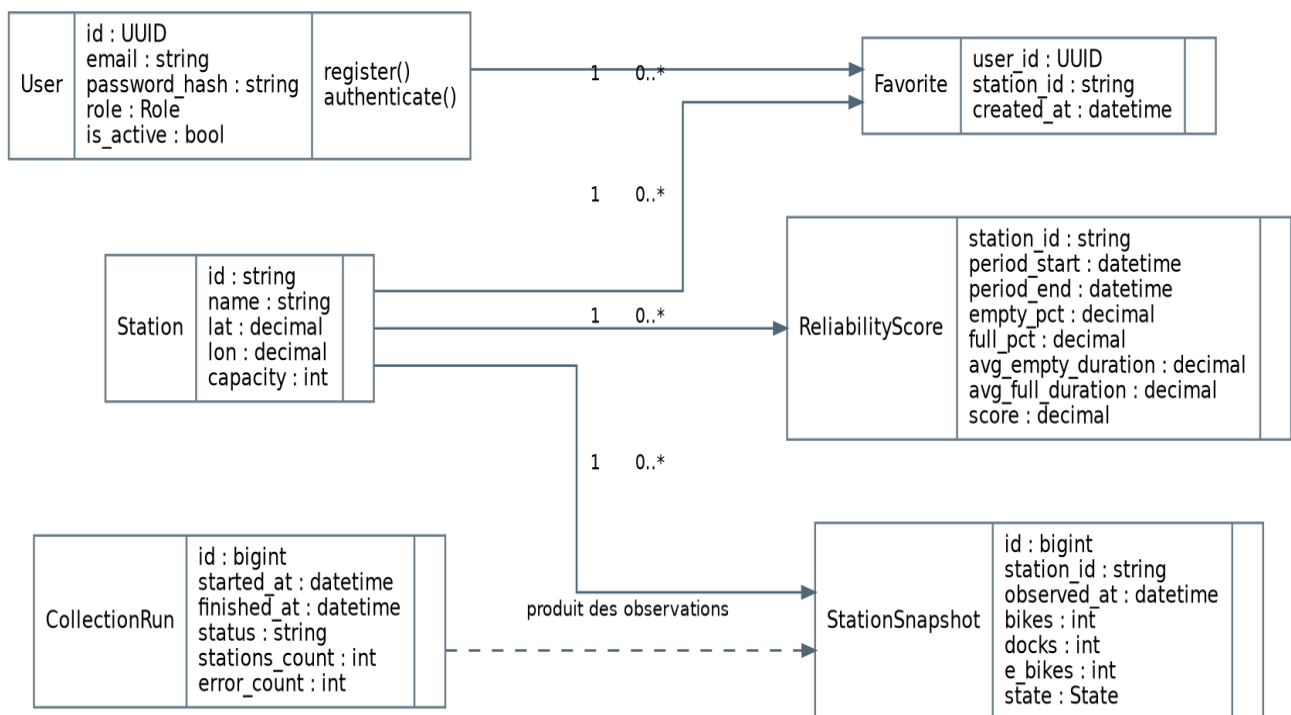


Figure 4 — Diagramme de classes principal.

Station représente le référentiel stable ; StationSnapshot représente une observation horodatée. User porte le compte et le rôle ; Favorite matérialise la relation utilisateur-station ; ReliabilityScore conserve les résultats analytiques ; CollectionRun trace les exécutions.

7. Conception de la base PostgreSQL

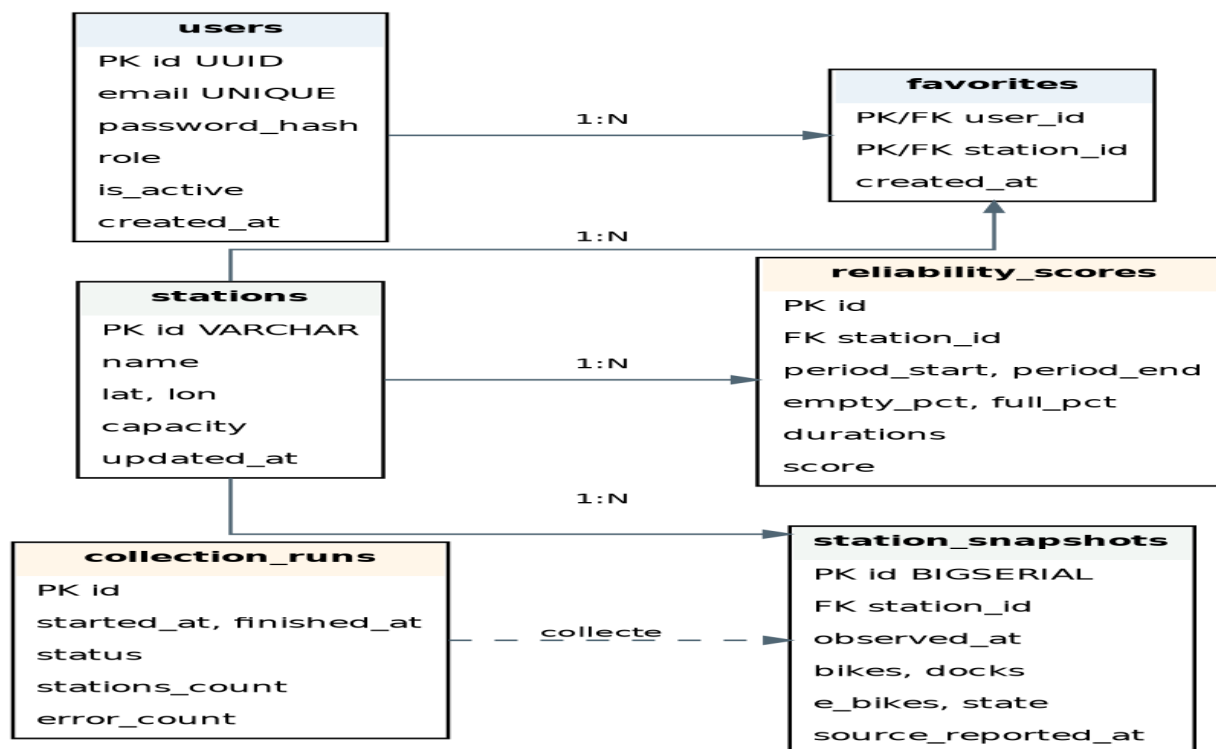


Figure 5 — Modèle relationnel.

Les tables principales sont users, stations, station_snapshots, favorites, reliability_scores et collection_runs. Les relations permettent de rattacher chaque observation à une station, chaque favori à un utilisateur et une station, et chaque score à une période.

L'historisation est append-only : une collecte ajoute des observations au lieu de remplacer les anciennes. L'index composite (station_id, observed_at DESC) est recommandé pour les requêtes d'historique.

8. Algorithmes métier

8.1 Classification

Pour chaque snapshot : EMPTY si bikes = 0 ; FULL si docks = 0 ; FUNCTIONAL sinon.

8.2 Épisodes

Un épisode commence lors de l'entrée dans EMPTY ou FULL et se termine à la sortie. Sa durée est calculée entre les timestamps. Les pourcentages de temps doivent idéalement tenir compte de la durée couverte par les observations.

8.3 Score de fiabilité

Score = $100 \times (1 - 0,55 \times \text{EMPTY_pct} - 0,45 \times \text{FULL_pct})$, borné dans [0,100]. Les coefficients sont un choix de projet : ils doivent être justifiés et testés.

8.4 Score de recommandation

RecommendationScore = $0,30 \times \text{DistanceScore} + 0,25 \times \text{AvailabilityScore} + 0,25 \times \text{ReliabilityScore} + 0,10 \times \text{EbikeScore} + 0,10 \times \text{TrendScore}$. Chaque critère est normalisé dans [0,1]. Deux stations à distance égale peuvent donc être classées différemment.

9. Diagrammes de séquence

Le diagramme de séquence décrit un scénario précis et l'ordre temporel des messages. Il complète l'architecture en montrant concrètement le déroulement d'une fonctionnalité.

9.1 Consultation de l'historique

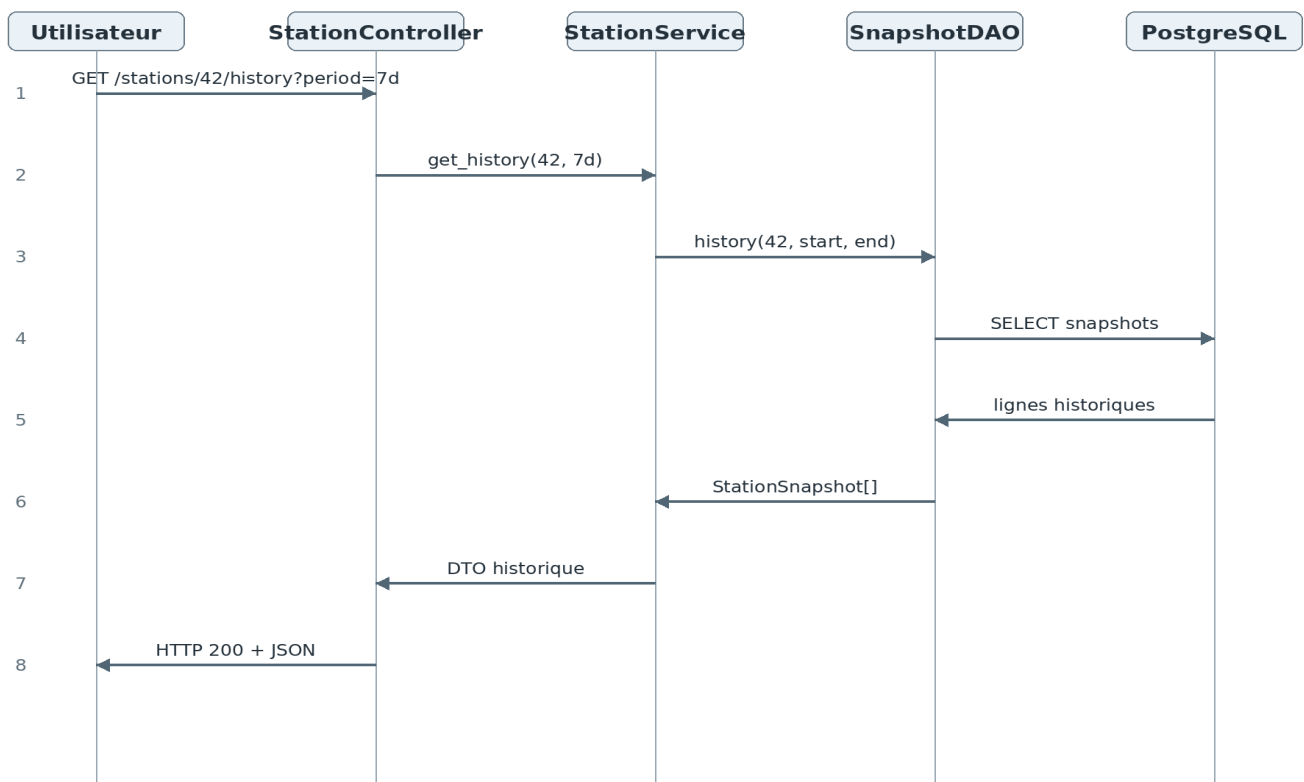


Figure 6 — Consultation de l'historique.

Le contrôleur valide la requête, le service définit la fenêtre temporelle, le DAO interroge PostgreSQL et la réponse remonte sous forme JSON.

9.2 Collecte périodique

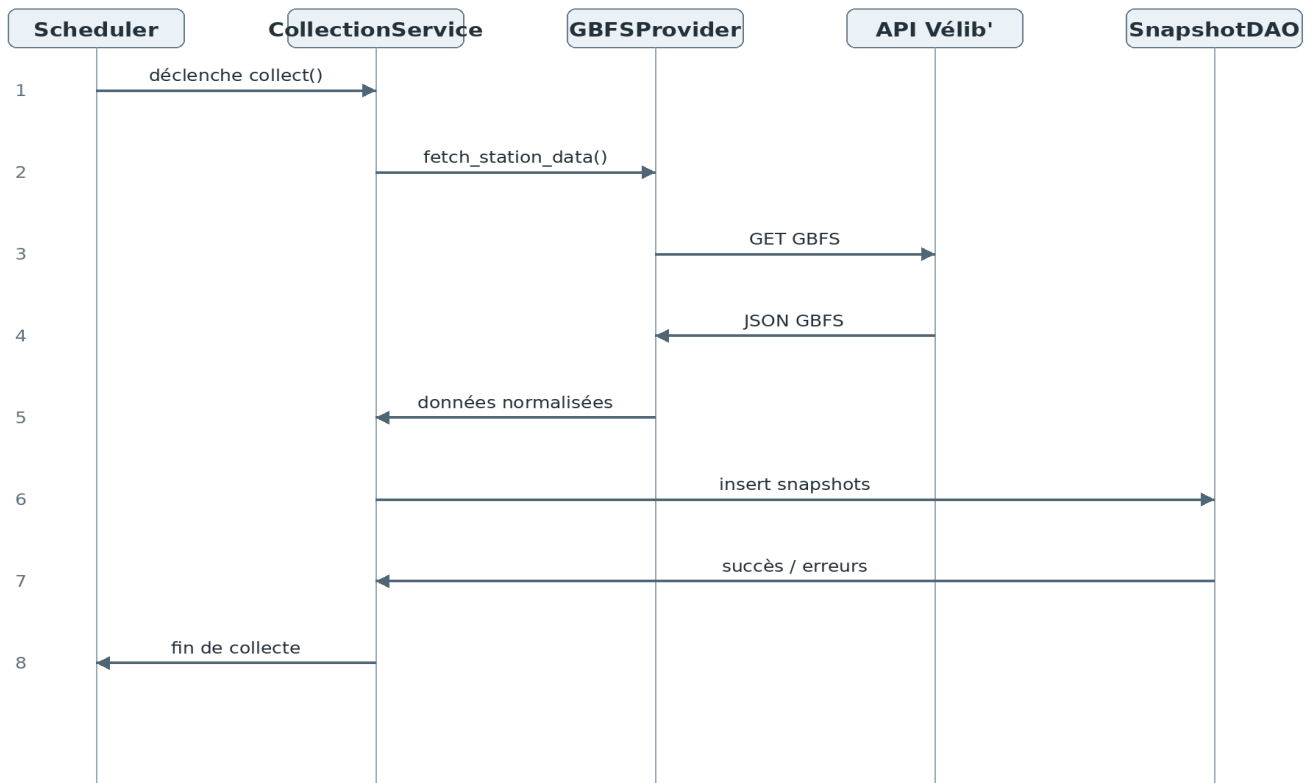


Figure 7 — Collecte périodique GBFS.

Le scheduler ne connaît pas les détails du fournisseur. Il déclenche CollectionService, qui délègue l'accès externe à GBFSProvider puis persiste les données normalisées via SnapshotDAO.

10. API REST

Méthode	Endpoint	Fonction
POST	/auth/register	Créer un compte
POST	/auth/login	Authentification
GET	/stations	Lister les stations
GET	/stations/{id}	État courant
GET	/stations/{id}/history	Historique
GET	/stations/{id}/reliability	Fiabilité
GET	/stations/recommendations	Recommandations
GET/POST/DELETE	/favorites...	Favoris
GET/PATCH	/admin/...	Administration

Les codes HTTP doivent être cohérents : 400 requête invalide, 401 authentification, 403 autorisation, 404 ressource absente et 500 erreur interne non anticipée.

11. Stratégie de tests

- Unitaire : fiabilité, recommandation, normalisation.
- Intégration : DAO, PostgreSQL, contraintes et transactions.
- API : scénarios nominaux et erreurs 400/401/403/404.
- GBFS mocké : timeout, erreur HTTP, JSON invalide, champ manquant.

Les tests de formules doivent couvrir station toujours vide, toujours pleine, toujours fonctionnelle, alternances d'états, absence de données, épisodes coupés par la fin de période et égalité de distance entre candidats.

12. Qualité et organisation

ruff contrôle le style et certaines erreurs statiques ; pytest automatise les tests ; Git assure le versionnement et la traçabilité.

Organisation recommandée :

app/controller/
app/service/
app/dao/
app/model/
app/integration/
app/scheduler/

tests/unit/
tests/integration/
tests/api/

13. Planification — diagramme de Gantt

Le Gantt présente une organisation indicative sur huit semaines. Les tâches peuvent être parallélisées dès que leurs dépendances sont satisfaites.

Tâche	S1	S2	S3	S4	S5	S6	S7	S8
Analyse / conception	■	■						
Architecture / PostgreSQL		■	■					
API FastAPI			■	■	■			
Collecte GBFS				■	■	■		
Consultation					■	■		
Fiabilité						■	■	
Recommandations							■	■
Favoris / Admin						■	■	
Tests						■	■	■
Documentation / livraison								■

Figure 8 — Planning indicatif.

Les jalons sont la validation de l'architecture, la première collecte persistée, la consultation historique, la validation de la fiabilité, la validation des recommandations puis la campagne finale de tests et de documentation.

14. Risques et perspectives

- Dépendance externe : l'API GBFS peut être indisponible ou évoluer ; l'adaptateur limite l'impact.
- Volume historique : la fréquence de collecte augmente rapidement le nombre de snapshots ; indexation et optimisation deviennent importantes.
- Qualité : les données externes doivent être validées avant insertion.
- Évolutivité : les calculs sur 30 jours peuvent nécessiter des optimisations avec la croissance du volume.

Les extensions FO1 à FO5 — prédiction, heatmap, anomalies, assistant de trajet et challenges — peuvent être ajoutées après stabilisation du cœur F1-F5. Leur intégration peut se faire par de nouveaux services et endpoints sans remettre en cause l'historisation.

15. Conclusion

La conception proposée transforme un flux GBFS courant en un service historique et analytique. L'architecture Controller → Service → DAO, complétée par GBFSProvider, sépare les responsabilités et rend les composants testables.

L'historisation par snapshots est le socle des statistiques, de la fiabilité et des recommandations. Les règles métier sont explicites, les formules documentées et les tests couvrent les cas nominaux comme les situations limites.

Résultat : une base logicielle cohérente, maintenable et suffisamment découplée pour évoluer vers les fonctionnalités optionnelles.